

乐器演奏器

1、实验任务：

应用硬件描述语言，设计一个乐曲硬件演奏电路，它能将一首预先设置存储好的乐曲自动播放出来，除此之外，也能够通过按键的方式输入音符，使其具备简易电子琴的功能。通过此项研究，能够深切的体会利用 EDA 工具开发的优越性，在此基础上，对乐曲硬件演奏电路功能进行丰富，具有一定的社会实用性。

2、实验原理：

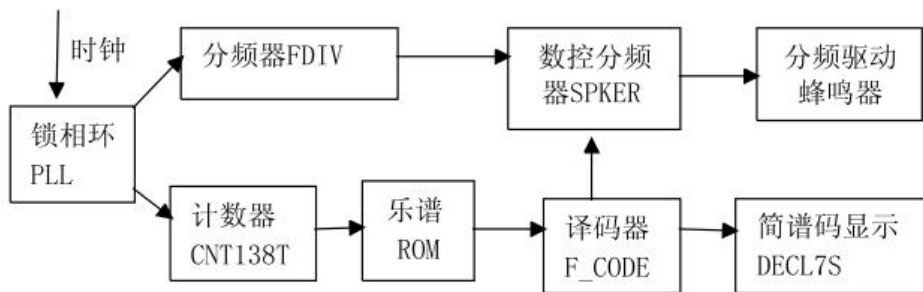


图 5-1 按键控制蜂鸣器系统框图

该工程由是如图 5-1 所示的 5 个模块构成

①输入的时钟 50MHZ 通过锁相环后输出 2KHZ 和 1KHZ 的时钟分别输送给分频器 FDIV 和计数器 CNT138;FDIV 将 1MHZ 的时钟分频成 4HZ 的时钟。

②计数器 CNT138T 完成计数功能, 138 进制，每来个时钟沿加一

③乐谱 ROM 记录了 1 首歌的乐谱。根据上一模块计数器所计的数读取相应地址里的数据传递给下一模块。

④译码器 FCODE 将选择器所选择的歌曲 rom 里的地址的数据翻译成可预置计数器 SPKER 和简谱码显示 DECL7S 所需的数据：

⑤分频驱动器也就是一个数控分频器，完成分频的功能，并驱动蜂鸣器。

当一个 4Hz 的时钟脉冲来到时，乐谱发生器模块输出一个音符数据给分频系数模块，分频系数模块输出此音符相应的分频系数所需的初始值，将初始值送给数控分频器模块，当 1MHz 的时钟脉冲来到时，数控分频器就根据分频系数输出相应的频率(即此音符所对应的发生频率)给扬声器，扬声器就可发出对应音符的声音来. 连续的 4Hz 的时钟脉冲就将乐谱发生器里所存储的音符数据一个接一个的送给了分频系数模块，再经过数控分频模块, 最后扬声器一个接一个的发出音符数据所对应的声音来。曲子也就流畅的播放出来了。同时 led 数码管会随着音乐显示相应的乐谱。当乐曲一遍演奏完成后，乐曲发生器能自动从头开始循环演奏。

硬件乐曲演奏电路 RTL 原理图如图 5-2 所示，与利用微处理器（CPU 或 MCU）来实

现乐曲演奏相比，以纯硬件完成乐曲演奏电路的逻辑要复杂一些。本实验设计项目作为《梁祝》乐曲演奏电路的实现。组成乐曲的每个音符的发音频率值及其持续的时间是乐曲能连续演奏所需的两个基本要素。问题是如何来获取这两个要素所对应的数值以及通过纯硬件的手段来利用这些数值实现所希望乐曲的演奏效果。下面首先从几个方面来了解图 5-2 的工作原理。

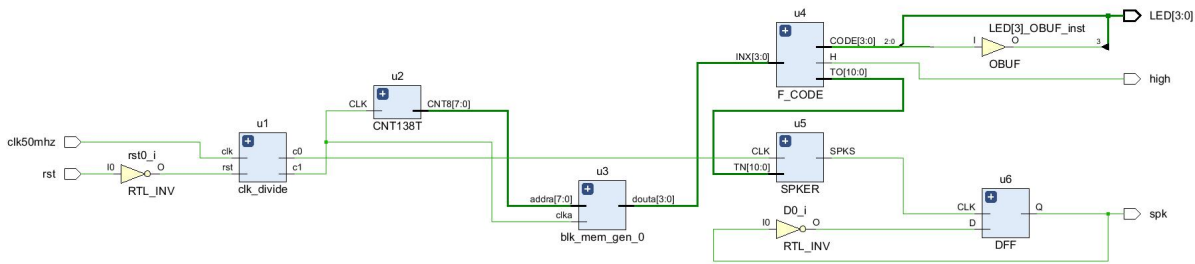


图 5-2 乐曲演奏电路顶层设计

(1) 音符的频率可以由图 5-3 中的 SPKER 获得，对应的程序如例 5-3 所示。这是一个用作分频器的可预置计数器。由 CLK 端输入一具有较高频率（1MHz）的时钟，通过 SPKER 分频后，经由 D 触发器构成的分频电路，由 SPK_KX 口输出。由于直接从分频器中出来的输出信号是脉宽极窄的信号，为了有利于驱动扬声器，需另加一个 D 触发器分频以均衡其占空比，但这时的频率将是原来的 1/2。SPKER 对 CLK 输入信号的分频比由输入的 11 位预置数 TN[10.0]决定。SPK_KX 的输出频率将决定每一音符的音调；这样，分频计数器的预置值 TN[10.0] 与输出频率就有了对应关系，而输出的频率又与音乐音符的发声有对应关系，例如，在 F_CODE 模块（例 5-2）中若取 TN[10.0]=11'H40C，将由 SPK_KX 发出音符为“3”音的信号频率。详细的对应关系可以参考图 5-3 的电子琴音阶基频对照图。

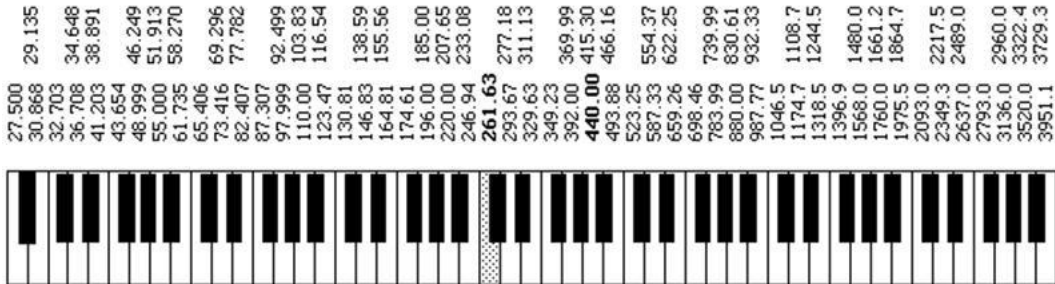


图 5-3 电子琴音阶基频对照图（单位：Hz）

(2) 音符的持续时间需根据乐曲的速度及每个音符的节拍数来确定，图 1.1 中模块 F_CODE（例 5-2）的功能首先是为模块 SPKER（11 位分频器）提供决定所发音符的分频预置数，而此数在 SPKER 输入口停留的时间即为此音符的节拍周期。模块 F_CODE 是乐曲简谱码对应的分频预置数查表电路，例 5-2 中的数据是根据图 5-3 得到的，程序中设置

了《梁祝》乐曲全部音符所对应的分频预置数，共 14 个，每一音符的停留时间则由音乐节拍和音调发生查表模块 MUSIC 中简谱码和工作时钟 inclock 的频率决定，在此为 4Hz。这 4Hz 频率来自分频模块 FDIV，模块 MUSIC 是一个 LPM_ROM。它的输入频率来自锁相环 PLL20 的 2kHz 输出频率。而模块 F_CODE 的 14 个值的输出由对应于 MUSIC 模块输出的 q[3..0] 及 4 位输入值 INX[3..0] 确定，而 INX[3..0] 最多有 16 种可选值。输向模块 F_CODE 中 INX[3..0] 的值在 SPKER 中对应的输出频率值与持续的时间由模块 MUSIC 决定。

(3) 模块 CNT138T 是一个 8 位二进制计数器，内部设置计数最大值为 139 (例 5-1)，作为音符数据 ROM 的地址发生器。这个计数器的计数频率即为 4Hz。即每一计数值的停留时间为 0.25 秒，即当全音符设为 1 秒时，4/4 拍的 4 分音符持续时间。例如，《梁祝》乐曲的第一个音符为“3”，此音在逻辑中停留了 4 个时钟节拍，即 1 秒时间，相应地，所对应的“3”音符分频预置值为 11'H40C，在 SPKER 的输入端停留了 1 秒。随着计数器 CNT138T 按 4Hz 的时钟速率作加法计数时，即随地址值递增时，音符数据 ROM 模块 MUSIC 中的音符数据将从 ROM 中通过 q[3..0] 端口输向 F_CODE 模块，《梁祝》乐曲就开始连续自然地演奏起来了。CNT138T 的节拍是 139，正好等于 ROM 中的简谱码数，所以可以确保循环演奏。对于其他乐曲，此计数最大值要根据情况更改。

定制音符数据 ROM MUSIC。该 ROM 中对应《梁祝》乐曲的音符数据已列于例 5-5 中。注意该例数据表中的数据位宽、深度和数据的表达类型。为了节省篇幅，例中的数据都横排了，实际上必须以每一分号为一行来展开。最后对该 ROM 进行仿真，确认例 5-5 中的音符数据已经进入 ROM 中。

3 、实验程序:

【例 5-1】

```
module CNT138T (CLK, CNT8);
input CLK; output[7:0] CNT8 ; reg[7:0] CNT; wire LD;
always @(posedge CLK or posedge LD ) begin
if (LD) CNT <= 8'b00000000 ; else CNT<=CNT+1; end
assign CNT8=CNT; assign LD=(CNT==138) ;
endmodule
```

【例 5-2】

```
module F_CODE (INX, CODE, H, T0);
input[3:0] INX; output[3:0] CODE; output H; output[10:0] T0;
reg[10:0] T0; reg[3:0] CODE; reg H;
always @(INX) begin
case (INX) //译码电路，查表方式，控制音调的预置
0 : begin T0 <= 11'H7FF; CODE<=0; H<=0; end
1 : begin T0 <= 11'H305; CODE<=1; H<=0; end
2 : begin T0 <= 11'H390; CODE<=2; H<=0; end
```

```

3 : begin TO <= 11'H40C; CODE<=3; H<=0; end
4 : begin TO <= 11'H45C; CODE<=4; H<=0; end
5 : begin TO <= 11'H4AD; CODE<=5; H<=0; end
6 : begin TO <= 11'H50A; CODE<=6; H<=0; end
7 : begin TO <= 11'H55C; CODE<=7; H<=0; end
8 : begin TO <= 11'H582; CODE<=1; H<=1; end
9 : begin TO <= 11'H5C8; CODE<=2; H<=1; end
10 : begin TO <= 11'H606; CODE<=3; H<=1; end
11 : begin TO <= 11'H640; CODE<=4; H<=1; end
12 : begin TO <= 11'H656; CODE<=5; H<=1; end
13 : begin TO <= 11'H684; CODE<=6; H<=1; end
14 : begin TO <= 11'H69A; CODE<=7; H<=1; end
15 : begin TO <= 11'H6C0; CODE<=1; H<=1; end
default : begin TO <= 11'H6C0; CODE<=1; H<=1;
end
endcase end
endmodule

```

【例 5-3】

```

module SPKER (CLK, TN, SPKS);
input CLK; input[10:0] TN; output SPKS;
reg SPKS; reg[10:0] CNT11;
always @(posedge CLK) begin : CNT11B_LOAD//11 位可预置计数器
if (CNT11==11'h7FF) begin CNT11=TN; SPKS<=1'b1; end
else begin CNT11=CNT11+1; SPKS<=1'b0 ; end
end
endmodule

```

【例 5-4】2kHz 至 4Hz 分频器

```

module FDIV (CLK,PM );
input CLK ; output PM ; reg [8:0] Q1; reg FULL; wire RST ;
always @(posedge CLK or posedge RST) begin
if (RST) begin Q1<=0; FULL<=1; end
else begin Q1 <= Q1+1; FULL<=0 ; end end
assign RST = ( Q1==499 ) ; assign PM = FULL ;
assign DOUT = Q1 ;endmodule

```

【例 5-5】ROM初始化.Coe乐谱文件

```

memory_initialization_radix=16;
memory_initialization_vector=3 3 3 3 5 5 5 6 8 8 8 9 6 8 5 5 C C C F D C
A C 9 9 9 9 9 9 0 9 9 9 A 7 7 6 6 5 5 5 6 8 8 9 9 3 3 8 8 6 5 6 8 5 5 5 5 5 5

```

```

5 A A A C 7 7 9 9 6 8 5 5 5 5 5 3 5 3 3 5 6 7 9 6 6 6 6 6 6 5 6 8 8 8 9 C C C A
9 9 A 9 8 8 6 5 3 3 3 3 8 8 8 8 6 8 6 5 3 5 6 8 5 5 5 5 5 5 5 5 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0;

```

4、实验仿真

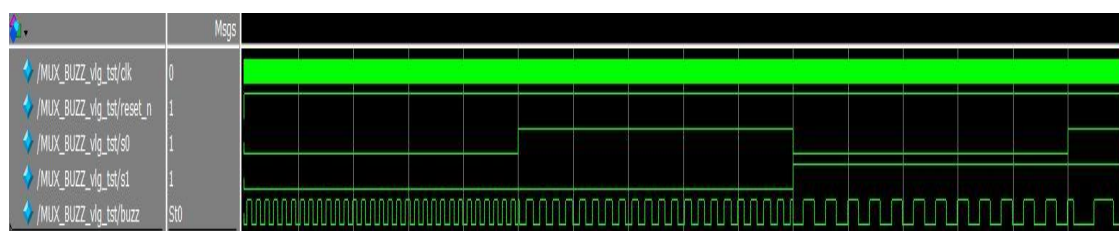


图 5-4

5、引脚约束:

#系统时钟约束

```
create_clock -period 20 -name clk [ get_ports clk50mhz ]
```

#IO 管脚约束

```

set_property -dict { PACKAGE_PIN Y18   IOSTANDARD LVCMOS33 } [get_ports clk50mhz]
set_property -dict { PACKAGE_PIN E3     IOSTANDARD LVCMOS33 } [get_ports rst]
set_property -dict { PACKAGE_PIN P20    IOSTANDARD LVCMOS33 } [get_ports spk ]
set_property -dict { PACKAGE_PIN AB18   IOSTANDARD LVCMOS33 } [get_ports high ]
set_property -dict { PACKAGE_PIN AA6    IOSTANDARD LVCMOS33 } [get_ports { LED[0] }]
set_property -dict { PACKAGE_PIN V7     IOSTANDARD LVCMOS33 } [get_ports { LED[1] }]
set_property -dict { PACKAGE_PIN W7     IOSTANDARD LVCMOS33 } [get_ports { LED[2] }]
set_property -dict { PACKAGE_PIN AB7    IOSTANDARD LVCMOS33 } [get_ports { LED[3] }]

```

6 、下载测试:

程序下载完毕后，将 SW5 开关拨上（BUZ）端。此时应该能够观察到开发板左边 4 位 LED1~LED4 灯显示当前音调的简谱码二进制码，右边一位 LED6 灯显示高八度指示，蜂鸣器演奏音乐《梁祝》。

